

Introduction

This program is a Transport Management System designed to manage buses, their schedules, and passengers. It allows users to add and manage details of buses, schedules, and passengers. The program is implemented in C and uses structures and linked lists to handle dynamic data efficiently. The program incorporates a restriction to ensure that no bus exceeds the capacity of 25 passengers. Before adding a new passenger, the program counts the current number of passengers for the bus. If the maximum limit is reached, it notifies the user and prevents further additions.

1.1 Key Features:

1. **Add Bus:**
 Allows adding a bus with a unique ID and driver name.
2. **Add Schedule to a Bus:**
 Users can add travel schedules (destination and time) for a specific bus.
3. **Add Passenger to a Bus:**
 Allows adding passengers to a specific bus, with a limit of 25 passengers per bus to ensure proper management.
4. **Display Details:**
 - View all buses.
 - View the schedules of a specific bus.
 - View the list of passengers on a specific bus.
5. **Delete Records:**
 - Delete a bus and all its associated schedules and passengers.
 - Remove a specific schedule from a bus.
 - Remove a specific passenger from a bus.
6. **Dynamic Memory Management:**
 The program uses dynamic memory allocation for schedules and passengers to handle an arbitrary number of entries efficiently.

This program serves as a basic transport management tool that can be extended further for additional features, such as fare calculation, route management, or integration with databases for persistence.

Project Design

The Bus Management System is a console-based application designed to manage bus information, schedules, and passenger data. The system allows administrators to add, delete, and display bus information, schedules, and passenger data.

Functional Requirements:

1. Bus Management:
 - Add bus information (bus ID, driver name)
 - Delete bus information
 - Display bus information
2. Schedule Management:
 - Add schedule information (destination, time) for a bus
 - Delete schedule information for a bus
 - Display schedule information for a bus
3. Passenger Management:
 - Add passenger information (name, age) for a bus
 - Delete passenger information for a bus
 - Display passenger information for a bus

System Design:

The system will consist of the following components:

1. Bus: Represents a bus with attributes (bus ID, driver name)
2. Schedule: Represents a schedule with attributes (destination, time)
3. Passenger: Represents a passenger with attributes (name, age)
4. Bus Array: An array of Bus structures to store bus information
5. Menu-driven Interface: A console-based interface to interact with the system

Data Structures:

1. Structures:
 - Bus
 - Schedule
 - Passenger

2. Arrays:

- Bus Array

3. Linked Lists:

- Schedule linked list for each bus
- Passenger linked list for each bus

Algorithms:

1. Add Bus: Add a new bus to the bus array
2. Delete Bus: Remove a bus from the bus array
3. Add Schedule: Add a new schedule to the schedule linked list for a bus
4. Delete Schedule: Remove a schedule from the schedule linked list for a bus
5. Add Passenger: Add a new passenger to the passenger linked list for a bus
6. Delete Passenger: Remove a passenger from the passenger linked list for a bus

Implementation:

The system will be implemented using C programming language.

Testing:

The system will be tested using test cases to ensure that it meets the functional requirements.

Deployment:

The system will be deployed on a console-based platform.

Maintenance:

The system will be maintained by updating the code to fix bugs and add new features as required.

Requirements

Here are the requirements for the Bus Management System program:

Functional Requirements:

1. Bus Management:
 - The system should allow administrators to add new buses.
 - The system should allow administrators to delete existing buses.
 - The system should display a list of all buses.
2. Schedule Management:
 - The system should allow administrators to add new schedules for each bus.
 - The system should allow administrators to delete existing schedules for each bus.
 - The system should display a list of all schedules for each bus.
3. Passenger Management:
 - The system should allow administrators to add new passengers for each bus.
 - The system should allow administrators to delete existing passengers for each bus.
 - The system should display a list of all passengers for each bus.

Non-Functional Requirements:

1. Performance:
 - The system should respond quickly to user input.
 - The system should be able to handle a large number of buses, schedules, and passengers.
2. Security:
 - The system should prevent unauthorized access to bus, schedule, and passenger data.
 - The system should ensure that all data is stored securely.
3. Usability:
 - The system should have a user-friendly interface.
 - The system should provide clear instructions and feedback to users.

Input/Output Requirements:

1. Input:
 - The system should accept input from users via a console-based interface.
 - The system should validate all user input to ensure it is correct and consistent.
2. Output:
 - The system should display output to users via a console-based interface.
 - The system should provide clear and concise output to users.

Data Requirements:

1. Data Storage:

- The system should store all bus, schedule, and passenger data in memory.
- The system should ensure that all data is stored securely.

2. Data Retrieval:

- The system should allow administrators to retrieve bus, schedule, and passenger data as needed.

Constraints:

1. Platform:

- The system should be developed using C programming language.
- The system should be compatible with console-based platforms.

2. Memory:

- The system should be able to run on systems with limited memory.

3. Time:

- The system should be able to handle a large number of transactions within a short period.

Limitations

Here are some limitations of the Transport Management System program:

Functional Limitations

1. Limited scalability: The program is designed to manage a small to medium-sized transportation system. It may not be suitable for large-scale transportation systems.
2. No real-time tracking: The program does not provide real-time tracking of vehicles, which may be a limitation for transportation systems that require real-time tracking.
3. No automated scheduling: The program does not have an automated scheduling feature, which may be a limitation for transportation systems that require complex scheduling.
4. No integration with other systems: The program does not integrate with other systems such as GPS tracking, traffic management, or payment systems.

Technical Limitations

1. Limited database capacity: The program uses a simple database that may not be suitable for large amounts of data.
2. No data encryption: The program does not encrypt data, which may be a security risk.
3. No backup and recovery: The program does not have a backup and recovery feature, which may result in data loss in case of a system failure.
4. Limited user authentication: The program does not have robust user authentication, which may be a security risk.

Usability Limitations

1. Steep learning curve: The program may have a steep learning curve for users who are not familiar with transportation management systems.
2. Limited user interface: The program's user interface may not be intuitive or user-friendly, which may result in errors or frustration.
3. No customization options: The program does not provide customization options, which may limit its flexibility.
4. No support for multiple languages: The program may not support multiple languages, which may limit its usability in diverse environments.

Maintenance Limitations

1. Limited maintainability: The program's code may not be maintainable, which may result in difficulties when updating or modifying the program.
2. No testing framework: The program does not have a testing framework, which may result in difficulties when testing the program.
3. No documentation: The program may not have documentation, which may result in difficulties when understanding the program's functionality.

Applications

Here are some potential applications of the Transport Management System program:

1.Transportation Industry

Public Transportation Systems: The program can be used to manage bus routes, schedules, and passenger information for public transportation systems.

Private Bus Operators: Private bus operators can use the program to manage their fleet, schedules, and passenger information.

Taxi and Ride-hailing Services: The program can be modified to manage taxi and ride-hailing services, including driver information, vehicle details, and passenger bookings.

2.Logistics and Supply Chain Management

Freight Management: The program can be used to manage freight operations, including vehicle scheduling, route optimization, and cargo tracking.

Warehouse Management: The program can be modified to manage warehouse operations, including inventory tracking, storage, and shipping.

3.Government Agencies

Transportation Departments: Government transportation departments can use the program to manage public transportation systems, including bus routes, schedules, and passenger information.

Traffic Management: The program can be used to manage traffic flow, including traffic signal control, traffic monitoring, and incident management.

4.Educational Institutions

University Transportation Systems: Universities can use the program to manage their transportation systems, including bus routes, schedules, and passenger information.

Student Transportation: The program can be used to manage student transportation, including bus schedules, route optimization, and student tracking.

5.Healthcare and Emergency Services

Ambulance Services: The program can be used to manage ambulance services, including vehicle scheduling, route optimization, and patient tracking.

Non-Emergency Medical Transportation: The program can be used to manage non-emergency medical transportation services, including vehicle scheduling, route optimization, and patient tracking.

6.Other Applications

Tour Operators: Tour operators can use the program to manage their fleet, schedules, and passenger information.

Corporate Transportation: Companies can use the program to manage their corporate transportation services, including employee transportation, vehicle scheduling, and route optimization.

IMPLEMENTATION OF THE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_BUSES 10
#define MAX_NAME_LENGTH 50

// Structures
typedef struct Schedule {
    char destination[MAX_NAME_LENGTH];
    char time[MAX_NAME_LENGTH];
    struct Schedule* next;
} Schedule;

typedef struct Passenger {
    char name[MAX_NAME_LENGTH];
    int age;
    struct Passenger* next;
} Passenger;

typedef struct Bus {
    int busID;
    char driverName[MAX_NAME_LENGTH];
    Schedule* scheduleHead;
    Passenger* passengerHead;
} Bus;

// Global Variables
Bus busArray[MAX_BUSES];
int busCount = 0;

// Function Prototypes
void addBus();
void addSchedule();
void addPassenger();
void displayBuses();
void displaySchedule(int busID);
void displayPassengers(int busID);
void deleteBus();
void deleteSchedule();
void deletePassenger();
void freeSchedules(Schedule* head);
void freePassengers(Passenger* head);
```

```
int main() {
    int choice;

    while (1) {
        printf("\n--- Transport Management System ---\n");
        printf("1. Add Bus\n");
        printf("2. Add Schedule to a Bus\n");
        printf("3. Add Passenger to a Bus\n");
        printf("4. Display Buses\n");
        printf("5. Display Schedule for a Bus\n");
        printf("6. Display Passengers for a Bus\n");
        printf("7. Delete a Bus\n");
        printf("8. Delete a Schedule from a Bus\n");
        printf("9. Delete a Passenger from a Bus\n");
        printf("10. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                addBus();
                break;
            case 2:
                addSchedule();
                break;
            case 3:
                addPassenger();
                break;
            case 4:
                displayBuses();
                break;
            case 5: {
                int busID;
                printf("Enter Bus ID: ");
                scanf("%d", &busID);
                displaySchedule(busID);
                break;
            }
            case 6: {
                int busID;
                printf("Enter Bus ID: ");
                scanf("%d", &busID);
                displayPassengers(busID);
                break;
            }
            case 7:
                deleteBus();
                break;
```

```
        case 8:
            deleteSchedule();
            break;
        case 9:
            deletePassenger();
            break;
        case 10:
            printf("Exiting...\n");
            return 0;
        default:
            printf("Invalid choice! Please try again.\n");
    }
}
}
```

// Function Definitions

```
void addBus() {
    if (busCount >= MAX_BUSES) {
        printf("Cannot add more buses. Maximum limit reached.\n");
        return;
    }

    Bus newBus;
    printf("Enter Bus ID: ");
    scanf("%d", &newBus.busID);
    printf("Enter Driver Name: ");
    scanf(" %[^\n]", newBus.driverName);
    newBus.scheduleHead = NULL;
    newBus.passengerHead = NULL;

    busArray[busCount++] = newBus;
    printf("Bus added successfully.\n");
}

void addSchedule() {
    int busID, found = 0;
    printf("Enter Bus ID: ");
    scanf("%d", &busID);

    for (int i = 0; i < busCount; i++) {
        if (busArray[i].busID == busID) {
            found = 1;
            Schedule* newSchedule = (Schedule*)malloc(sizeof(Schedule));
            if (!newSchedule) {
                printf("Memory allocation failed.\n");
                return;
            }
        }
    }
```

```
        printf("Enter Destination: ");
        scanf(" %[^\\n]", newSchedule->destination);
        printf("Enter Time: ");
        scanf(" %[^\\n]", newSchedule->time);
        newSchedule->next = busArray[i].scheduleHead;
        busArray[i].scheduleHead = newSchedule;
        printf("Schedule added successfully.\\n");
        break;
    }
}

if (!found) {
    printf("Bus with ID %d not found.\\n", busID);
}
}

void addPassenger() {
    int busID, found = 0;
    printf("Enter Bus ID: ");
    scanf("%d", &busID);

    for (int i = 0; i < busCount; i++) {
        if (busArray[i].busID == busID) {
            found = 1;

            // Count current passengers
            int passengerCount = 0;
            Passenger* temp = busArray[i].passengerHead;
            while (temp) {
                passengerCount++;
                temp = temp->next;
            }

            // Check if limit is reached
            if (passengerCount >= 25) {
                printf("Cannot add more passengers. Maximum limit of 25 reached.\\n");
                return;
            }

            // Add new passenger
            Passenger* newPassenger = (Passenger*)malloc(sizeof(Passenger));
            if (!newPassenger) {
                printf("Memory allocation failed.\\n");
                return;
            }
            printf("Enter Passenger Name: ");
            scanf(" %[^\\n]", newPassenger->name);
            printf("Enter Passenger Age: ");
```

```
        scanf("%d", &newPassenger->age);
        newPassenger->next = busArray[i].passengerHead;
        busArray[i].passengerHead = newPassenger;
        printf("Passenger added successfully.\n");
        return;
    }
}

if (!found) {
    printf("Bus with ID %d not found.\n", busID);
}
}

void displayBuses() {
    if (busCount == 0) {
        printf("No buses available.\n");
        return;
    }

    printf("\n--- List of Buses ---\n");
    for (int i = 0; i < busCount; i++) {
        printf("Bus ID: %d, Driver Name: %s\n", busArray[i].busID, busArray[i].driverName);
    }
}

void displaySchedule(int busID) {
    for (int i = 0; i < busCount; i++) {
        if (busArray[i].busID == busID) {
            printf("\n--- Schedule for Bus ID %d ---\n", busID);
            Schedule* temp = busArray[i].scheduleHead;
            if (!temp) {
                printf("No schedules available for this bus.\n");
            } else {
                while (temp) {
                    printf("Destination: %s, Time: %s\n", temp->destination, temp->time);
                    temp = temp->next;
                }
            }
            return;
        }
    }
    printf("Bus with ID %d not found.\n", busID);
}

void displayPassengers(int busID) {
    for (int i = 0; i < busCount; i++) {
        if (busArray[i].busID == busID) {
            printf("\n--- Passengers for Bus ID %d ---\n", busID);
            Passenger* temp = busArray[i].passengerHead;
```

```
        if (!temp) {
            printf("No passengers available for this bus.\n");
        } else {
            while (temp) {
                printf("Name: %s, Age: %d\n", temp->name, temp->age);
                temp = temp->next;
            }
        }
        return;
    }
}
printf("Bus with ID %d not found.\n", busID);
}
```

```
void deleteBus() {
    int busID, found = 0;
    printf("Enter Bus ID to delete: ");
    scanf("%d", &busID);

    for (int i = 0; i < busCount; i++) {
        if (busArray[i].busID == busID) {
            found = 1;
            freeSchedules(busArray[i].scheduleHead);
            freePassengers(busArray[i].passengerHead);
            for (int j = i; j < busCount - 1; j++) {
                busArray[j] = busArray[j + 1];
            }
            busCount--;
            printf("Bus deleted successfully.\n");
            break;
        }
    }

    if (!found) {
        printf("Bus with ID %d not found.\n", busID);
    }
}
```

```
void deleteSchedule() {
    int busID, found = 0;
    char destination[MAX_NAME_LENGTH];
    printf("Enter Bus ID: ");
    scanf("%d", &busID);

    for (int i = 0; i < busCount; i++) {
        if (busArray[i].busID == busID) {
            found = 1;
            printf("Enter Destination to delete: ");
        }
    }
}
```

```
scanf("%[^\n]", destination);
Schedule *current = busArray[i].scheduleHead, *prev = NULL;
while (current) {
    if (strcmp(current->destination, destination) == 0) {
        if (prev) {
            prev->next = current->next;
        } else {
            busArray[i].scheduleHead = current->next;
        }
        free(current);
        printf("Schedule deleted successfully.\n");
        return;
    }
    prev = current;
    current = current->next;
}
printf("Destination not found.\n");
break;
}
}

if (!found) {
    printf("Bus with ID %d not found.\n", busID);
}
}

void deletePassenger() {
    int busID, found = 0;
    char name[MAX_NAME_LENGTH];
    printf("Enter Bus ID: ");
    scanf("%d", &busID);

    for (int i = 0; i < busCount; i++) {
        if (busArray[i].busID == busID) {
            found = 1;
            printf("Enter Passenger Name to delete: ");
            scanf("%[^\n]", name);
            Passenger *current = busArray[i].passengerHead, *prev = NULL;
            while (current) {
                if (strcmp(current->name, name) == 0) {
                    if (prev) {
                        prev->next = current->next;
                    } else {
                        busArray[i].passengerHead = current->next;
                    }
                    free(current);
                    printf("Passenger deleted successfully.\n");
                    return;
                }
            }
        }
    }
}
```

```
        }
        prev = current;
        current = current->next;
    }
    printf("Passenger not found.\n");
    break;
}
}

if (!found) {
    printf("Bus with ID %d not found.\n", busID);
}
}

void freeSchedules(Schedule* head) {
    Schedule* temp;
    while (head) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

void freePassengers(Passenger* head) {
    Passenger* temp;
    while (head) {
        temp = head;
        head = head->next;
        free(temp);
    }
}
```


Results

--- Transport Management System ---

1. Add Bus
2. Add Schedule to a Bus
3. Add Passenger to a Bus
4. Display Buses
5. Display Schedule for a Bus
6. Display Passengers for a Bus
7. Delete a Bus
8. Delete a Schedule from a Bus
9. Delete a Passenger from a Bus
10. Exit

Enter your choice: 1

Enter Bus ID: 101

Enter Driver Name: John Doe

Bus added successfully.

--- Transport Management System ---

1. Add Bus
2. Add Schedule to a Bus
3. Add Passenger to a Bus
4. Display Buses
5. Display Schedule for a Bus
6. Display Passengers for a Bus
7. Delete a Bus
8. Delete a Schedule from a Bus
9. Delete a Passenger from a Bus
10. Exit

Enter your choice: 2

Enter Bus ID: 101

Enter Destination: New York

Enter Time: 10:00 AM

Schedule added successfully.

--- Transport Management System ---

1. Add Bus
2. Add Schedule to a Bus
3. Add Passenger to a Bus
4. Display Buses
5. Display Schedule for a Bus
6. Display Passengers for a Bus
7. Delete a Bus
8. Delete a Schedule from a Bus
9. Delete a Passenger from a Bus
10. Exit

Enter your choice: 3

Enter Bus ID: 101

Enter Passenger Name: Jane Doe

Enter Passenger Age: 30

Passenger added successfully.

--- Transport Management System ---

1. Add Bus
2. Add Schedule to a Bus
3. Add Passenger to a Bus
4. Display Buses
5. Display Schedule for a Bus
6. Display Passengers for a Bus
7. Delete a Bus
8. Delete a Schedule from a Bus
9. Delete a Passenger from a Bus
10. Exit

Enter your choice: 4

--- List of Buses ---

Bus ID: 101, Driver Name: John Doe

--- Transport Management System ---

1. Add Bus
2. Add Schedule to a Bus
3. Add Passenger to a Bus
4. Display Buses
5. Display Schedule for a Bus
6. Display Passengers for a Bus

7. Delete a Bus
8. Delete a Schedule from a Bus
9. Delete a Passenger from a Bus
10. Exit

Enter your choice: 5

Enter Bus ID: 101

--- Schedule for Bus ID 101 ---

Destination: New York, Time: 10:00 AM

--- Transport Management System ---

1. Add Bus
2. Add Schedule to a Bus
3. Add Passenger to a Bus
4. Display Buses
5. Display Schedule for a Bus
6. Display Passengers for a Bus
7. Delete a Bus
8. Delete a Schedule from a Bus
9. Delete a Passenger from a Bus
10. Exit

Enter your choice: 6

Enter Bus ID: 101

--- Passengers for Bus ID 101 ---

Name: Jane Doe, Age: 30

--- Transport Management System ---

1. Add Bus
2. Add Schedule to a Bus
3. Add Passenger to a Bus
4. Display Buses
5. Display Schedule for a Bus
6. Display Passengers for a Bus
7. Delete a Bus
8. Delete a Schedule from a Bus
9. Delete a Passenger from a Bus
10. Exit

Enter your choice: 10

Future Enhancements

1. User Authentication and Authorization

Implement user authentication to ensure that only authorized personnel can access and modify data.

Create different user roles with varying levels of access and permissions.

2. Data Analytics and Reporting

Integrate data analytics capabilities to provide insights into bus operations, passenger trends, and revenue generation.

Generate reports on key performance indicators (KPIs) such as on-time performance, passenger satisfaction, and revenue growth.

3. Mobile App Integration

Develop a mobile app for passengers to access bus schedules, book tickets, and receive real-time updates.

Integrate mobile payment gateways for seamless ticket booking and payment.

4. Real-Time Tracking and Monitoring

Implement real-time tracking and monitoring of buses using GPS and IoT technologies.

Provide real-time updates on bus locations, arrival times, and traffic conditions.

5. Automated Scheduling and Optimization

Develop an automated scheduling system that optimizes bus routes and schedules based on passenger demand and traffic patterns.

Integrate machine learning algorithms to predict passenger demand and adjust schedules accordingly.

6. Integration with Other Systems

Integrate the Transport Management System with other systems such as traffic management systems, emergency services, and public transportation systems.

Develop APIs for seamless integration with other systems and applications.

Test Cases

Test Case 1: Add Bus

- Input:
 - Choose option 1 (Add Bus)
 - Enter Bus ID: 101
 - Enter Driver Name: John Doe
- Expected Output:
 - "Bus added successfully."
 - Bus ID: 101, Driver Name: John Doe should be displayed in the list of buses

Test Case 2: Add Schedule

- Input:
 - Choose option 2 (Add Schedule to a Bus)
 - Enter Bus ID: 101
 - Enter Destination: New York
 - Enter Time: 10:00 AM
- Expected Output:
 - "Schedule added successfully."
 - Destination: New York, Time: 10:00 AM should be displayed in the schedule for Bus ID 101

Test Case 3: Add Passenger

- Input:
 - Choose option 3 (Add Passenger to a Bus)
 - Enter Bus ID: 101
 - Enter Passenger Name: Jane Doe
 - Enter Passenger Age: 30
- Expected Output:
 - "Passenger added successfully."
 - Name: Jane Doe, Age: 30 should be displayed in the list of passengers for Bus ID 101

Test Case 4: Display Buses

- Input:
 - Choose option 4 (Display Buses)
- Expected Output:
 - A list of all buses, including their IDs and driver names

Test Case 5: Display Schedule

- Input:
 - Choose option 5 (Display Schedule for a Bus)
 - Enter Bus ID: 101

- Expected Output:
 - A list of all schedules for Bus ID 101, including destinations and times

Test Case 6: Display Passengers

- Input:
 - Choose option 6 (Display Passengers for a Bus)
 - Enter Bus ID: 101
- Expected Output:
 - A list of all passengers for Bus ID 101, including their names and ages

Test Case 7: Delete Bus

- Input:
 - Choose option 7 (Delete a Bus)
 - Enter Bus ID to delete: 101
- Expected Output:
 - "Bus deleted successfully."
 - Bus ID 101 should be removed from the list of buses

Test Case 8: Delete Schedule

- Input:
 - Choose option 8 (Delete a Schedule from a Bus)
 - Enter Bus ID: 101
 - Enter Destination to delete: New York
- Expected Output:
 - "Schedule deleted successfully."
 - Destination: New York should be removed from the schedule for Bus ID 101

Test Case 9: Delete Passenger

- Input:
 - Choose option 9 (Delete a Passenger from a Bus)
 - Enter Bus ID: 101
 - Enter Passenger Name to delete: Jane Doe
- Expected Output:
 - "Passenger deleted successfully."
 - Name: Jane Doe should be removed from the list of passengers for Bus ID 101

Test Case 10: Exit

- Input:
 - Choose option 10 (Exit)
- Expected Output:
 - "Exiting..."
 - The program should terminate.

Conclusion

the Transport Management System program is a powerful and versatile application that has the potential to transform the way transport managers work. Its comprehensive feature set, intuitive interface, and emphasis on data validation and error handling make it an ideal solution for transport organizations seeking to improve their operational efficiency and effectiveness. Whether you are a transport manager, a business owner, or simply someone who is passionate about transportation, this program is definitely worth exploring. The future of transportation management is here, and it is brighter than ever. With the Transport Management System program, transport managers can look forward to a future where data management is streamlined, decision-making is informed, and operational efficiency is optimized. So why wait? Embark on the journey towards transportation management excellence today. In the ever-evolving landscape of transportation management, it is essential to stay ahead of the curve. The Transport Management System program is more than just a software application – it is a game-changer. It is a testament to the power of technology in transforming the way we work and live. So, what are you waiting for? Experience the future of transportation management today

References

Books:

1. "The C Programming Language" by Brian Kernighan and Dennis Ritchie
2. "C: How to Program" by Paul Deitel and Harvey Deitel
3. "Data Structures and Algorithm Analysis in C" by Mark Allen Weiss

Online Resources:

1. GeeksforGeeks: C Programming Language
2. Tutorialspoint: C Programming Tutorial
3. Codecademy: C Programming Course
4. Stack Overflow: C Programming Questions and Answers